Topic 4 Notes Introduction

Introduction

The software requirements are description of features and functionalities of the target system. Requirements convey the expectations of users from the software product. The requirements can be obvious or hidden, known or unknown, expected or unexpected from client's point of view.

Requirement Engineering

The process to gather the software requirements from client, analyze and document them is known as requirement engineering.

The goal of requirement engineering is to develop and maintain sophisticated and descriptive 'System Requirements Specification' document.

Requirement Engineering Process

It is a four step process, which includes –

Feasibility Study

- Requirement Gathering
- Software Requirement Specification
- Software Requirement Validation

Let us see the process briefly - **Feasibility study**

When the client approaches the organization for getting the desired product developed, it comes up with rough idea about what all functions the software must perform and which all features are expected from the software.

Referencing to this information, the analysts does a detailed study about whether the desired system and its functionality are feasible to develop.

This feasibility study is focused towards goal of the organization. This study analyzes whether the software product can be practically materialized in terms of implementation, contribution of project to organization, cost constraints and as per values and objectives of the organization. It explores technical aspects of the project and product such as usability, maintainability, productivity and integration ability.

The output of this phase should be a feasibility study report that should contain adequate comments and recommendations for management about whether or not the project should be undertaken.

Requirement Gathering

If the feasibility report is positive towards undertaking the project, next phase starts with gathering requirements from the user. Analysts and engineers communicate with the client and end-users to know their ideas on what the software should provide and which features they want the software to include.

Software Requirement Specification

SRS is a document created by system analyst after the requirements are collected from various stakeholders.

SRS defines how the intended software will interact with hardware, external interfaces, speed of operation, response time of system, portability of software across various platforms, maintainability, speed of recovery after crashing, Security, Quality, Limitations etc.

The requirements received from client are written in natural language. It is the responsibility of system analyst to document the requirements in technical language so that they can be comprehended and useful by the software development team.
SRS should come up with following features:

User Requirements are expressed in natural language.

- Technical requirements are expressed in structured language, which is used inside the organization.
- Design description should be written in Pseudo code.
- Format of Forms and GUI screen prints.
- Conditional and mathematical notations for DFDs etc.

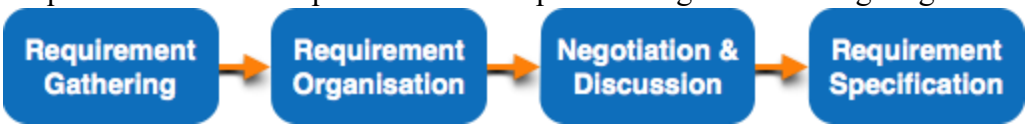
Software Requirement Validation

After requirement specifications are developed, the requirements mentioned in this document are validated. User might ask for illegal, impractical solution or experts may interpret the requirements incorrectly. This results in huge increase in cost if not nipped in the bud. Requirements can be checked against following conditions - If they can be practically implemented

- If they are valid and as per functionality and domain of software
- If there are any ambiguities
- If they are complete
- If they can be demonstrated

Requirement Elicitation Process

Requirement elicitation process can be depicted using the folloiwng diagram:



- **Requirements gathering -** The developers discuss with the client and end users and know their expectations from the software.
- **Organizing Requirements -** The developers prioritize and arrange the requirements in order of importance, urgency and convenience.
- **Negotiation & discussion -** If requirements are ambiguous or there are some conflicts in requirements of various stakeholders, if they are, it is then negotiated and discussed with stakeholders. Requirements may then be prioritized and reasonably compromised. The requirements come from various stakeholders. To remove the ambiguity and conflicts, they are discussed for clarity and correctness. Unrealistic requirements are compromised reasonably.
- **Documentation -** All formal & informal, functional and non-functional requirements are documented and made available for next phase processing.

The Process for Requirements Collection

Typically, requirements gathering is made up a few discrete steps.

Appoint and Assign

First things first: who's going to be the person that tells everyone you're the project manager? Ensure that that person understands why this role is so important—everyone must go to you with all project updates, as you will act as the knowledge center for project progress.

You'll also want to identify who the key stakeholders will be. These will be the people who brainstorm, analyze, approve or deny project updates. They're typically made up of customers, team leads, department managers, board members, business partners or manufacturers. They'll have the most say in the progress of the project overall.

Elicit Requirements

Next, you'll want to interview all of the stakeholders that you identified. Ask them questions like:

- What is on your wishlist for this product update?
- What is your ultimate goal for this project?
- What do you wish this product would do that it doesn't already?
- What got you interested in this product in the first place?
- What changes would convince you to recommend this product to others?
- What tools would you need to make this project successful?
- What are the concerns you have for this project process?

Gather and Document

Record every single answer, and create an easily-accessible repository where (approved) others can access if they need to reference any information that was collected during the requirements gathering phase.

Not only will this documentation be helpful at the end of the project when you reflect back on goals achieved, updates accomplished, features added and bugs fixed, it will also act to help manage stakeholder expectations, and keep team members focused and on track.

List All Assumptions and Requirements

Once you've documented everyone's goals and expectations, you can create a requirements management plan that's actionable, measurable and quantifiable. During this phase you'll answer:

- **How long will the project timeline be?** Map out your timeline, and then map out your requirements on that timeline. This will help in case some requirements are contingent on dependencies.
- **Who will be involved in the project?** Will it be the entire design and development teams, or just a select few from each? Which team members will be available? Which team members specialize in the types of issues the project will tackle?
- **What are the risks for the requirements gathering process?** Define all assumptions, and document all risks that might impact your requirements. Understand that your assumptions are typically divided into three categories: time, budget and scope. They can range from assuming PTO, holidays and sick days, to assuming stakeholders will provide feedback in a timely manner.

- **What is our ultimate goal in understanding our project requirements?** What is the time-based goal, the budget goal and the scope goal? Will it be to compete in the market more directly with a competitor? Will it be to solve a customer problem, or fix a bug?

By answering all of the questions above in a clear and concise manner, you'll have a full map of your requirements ready to present to stakeholders.

Monitor Progress

Once you've gotten stakeholder approval on the requirements you've presented, you'll implement them into the project timeline and process. At this point, you'll want to make sure you have a method in place to monitor and track all of your requirements across all teams to ensure that triggers for risk stays low.

You'll also want to use this data to report project progress to stakeholders, give feedback to department managers, and ensure the project is on track from a time, scope and budget standpoint.

Tips for Successful Requirements Collection

Following are some of the tips for making the requirements collection process successful:

- Never assume that you know customer's requirements. What you usually think, could be quite different to what the customer wants. Therefore, always verify with the customer when you have an assumption or a doubt.
- Get the end-users involved from the start. Get their support for what you do.
- At the initial levels, define the scope and get customer's agreement. This helps you to successfully focus on scope of features.
- When you are in the process of collecting the requirements, make sure that the requirements are realistic, specific and measurable.
- Focus on making the requirements document crystal clear. Requirement document is the only way to get the client and the service provider to an agreement. Therefore, there should not be any gray area in this document. If there are gray areas, consider this would lead to potential business issues.
- Do not talk about the solution or the technology to the client until all the requirements are gathered. You are not in a position to promise or indicate anything to the client until you are clear about the requirements.
- Before moving into any other project phases, get the requirements document signed off by the client.
- If necessary, create a prototype to visually illustrate the requirements.

Categories of Software Requirements

We should try to understand what sort of requirements may arise in the requirement elicitation phase and what kinds of requirements are expected from the software system.

Broadly software requirements should be categorized in two categories: **Functional Requirements**

Requirements, which are related to functional aspect of software fall into this category. They define functions and functionality within and from the software system.

Examples -

- Search option given to user to search from various invoices.
- User should be able to mail any report to management.
- Users can be divided into groups and groups can be given separate rights.
- Should comply business rules and administrative functions.

Software is developed keeping downward compatibility intact.

Non-Functional Requirements

Requirements, which are not related to functional aspect of software, fall into this category. They are implicit or expected characteristics of software, which users make assumption of.

Non-functional requirements include -

- Security
- Logging
- Storage
- Configuration
- Performance
- Cost
- Interoperability
- Flexibility
- Disaster recovery
- Accessibility

Requirements are categorized logically as

- **Must Have** : Software cannot be said operational without them.
- **Should have** : Enhancing the functionality of software.
- **Could have** : Software can still properly function with these requirements.
- **Wish list** : These requirements do not map to any objectives of software.

While developing software, 'Must have' must be implemented, 'Should have' is a matter of debate with stakeholders and negation, whereas 'could have' and 'wish list' can be kept for software updates.

User Interface requirements

UI is an important part of any software or hardware or hybrid system. A software is widely accepted if it is -

- easy to operate
- quick in response
- effectively handling operational errors
- providing simple yet consistent user interface

User acceptance majorly depends upon how user can use the software. UI is the only way for users to perceive the system. A well performing software system must also be equipped with attractive, clear, consistent and responsive user interface. Otherwise the functionalities of software system can not be used in convenient way. A system is said be good if it provides means

to use it efficiently. User interface requirements are briefly mentioned below - Content presentation

- Easy Navigation
- Simple interface
- Responsive
- Consistent UI elements
- Feedback mechanism
- Default settings
- Purposeful layout
- Strategical use of color and texture. Provide help information

User centric approach
Group based view settings.

Software requirements specification (SRS)

A software requirements specification (SRS) is a comprehensive description of the intended purpose and environment for software under development. The SRS fully describes what the software will do and how it will be expected to perform.

An SRS minimizes the time and effort required by developers to achieve desired goals and also minimizes the development cost.

A good SRS defines how an application will interact with system hardware, other programs and human users in a wide variety of real-world situations. Parameters such as operating speed, response time, availability, portability, maintainability, footprint, security and speed of recovery from adverse events are evaluated. Methods of defining an SRS are described by the IEEE (Institute of Electrical and Electronics Engineers) specification 830-1998.

Key components of an SRS

The main sections of a software requirements specification are:

- **Business drivers** – this section describes the reasons the customer is looking to build the system, including problems with the currently system and opportunities the new system will provide.
- **Business model** – this section describes the business model of the customer that the system has to support, including organizational, business context, main business functions and process flow diagrams.
- **Business/functional and system requirements** -- this section typically consists of requirements that are organized in a hierarchical structure. The business/functional requirements are at the top level and the detailed system requirements are listed as child items.
- **Business and system use cases** -- this section consists of a Unified Modeling Language (UML) use case diagram depicting the key external entities that will be interacting with the system and the different use cases that they'll have to perform.
- **Technical requirements** -- this section lists the non-functional requirements that make up the technical environment where software needs to operate and the technical restrictions under which it needs to operate.

- **System qualities** -- this section is used to describe the non-functional requirements that define the quality attributes of the system, such as reliability, serviceability, security, scalability, availability and maintainability.
- **Constraints and assumptions** -- this section includes any constraints that the customer has imposed on the system design. It also includes the requirements engineering team's assumptions about what is expected to happen during the project.
- **Acceptance criteria** -- this section details the conditions that must be met for the customer to accept the final system.

Purpose of an SRS

An SRS forms the basis of an organization's entire project. It sets out the framework that all the development teams will follow. It provides critical information to all the teams, including

development, operations, quality assurance (QA) and maintenance, ensuring the teams are in agreement.

Using the SRS helps an enterprise confirm that the requirements are fulfilled and helps business leaders make decisions about the lifecycle of their product, such as when to retire a feature. In addition, writing an SRS can help developers reduce the time and effort necessary to meet their goals as well as save money on the cost of development.

SRS Structure

The following is a simple SRS structure:

Table of Contents

Features of an SRS

An SRS should have following characteristics:

- **Correct** -- should accurately reflect product functionality and specification at any point of time.
- **Unambiguous --** should not be any confusion regarding interpretation of the requirements. **Complete** -- should contain all the features requested by a client.
- **Consistent** -- same abbreviation and conventions must be followed throughout the document.
- **Ranked for importance and/or stability** -- every requirement is important. But some are urgent and must be fulfilled before other requirements and some could be delayed. It's better to classify each requirement according to its importance and stability.
- **Verifiable** -- an SRS is verifiable only if every stated requirement can be verified. A requirement is verifiable if there is some method to quantifiably measure whether the final software meets that requirement.
- **Modifiable --** an SRS must clearly identify each and every requirement in a systematic manner. If there are any changes, the specific requirements and the dependent ones can be modified accordingly without impact the others.
- **Traceable** – an SRS is traceable if the origin of each of its requirements is clear and if it makes it easy to reference each requirement in future development.

The goals of an SRS

Some of the goals an SRS should achieve are to:

- Provide feedback to the customer, ensuring that the IT company understands the issues the software system should solve and how to address those issues.
- Help to break a problem down into smaller components just by writing down the requirements.
- Speed up the testing and validation processes.
- Facilitate reviews

Requirement Gathering Methods

One-on-One Interviews

One-on-one interviews are the most common technique for gathering requirements, as well as one of the primary sources of requirements. To help get the most out of an interview, they should be well thought out and prepared before sitting with the interviewee. The analyst should identify stakeholders to be interviewed. These can be users who interact with the current or new system, management, project financers or anyone else that would be involved in the system. When preparing an interview is it important to ask open-ended questions, as well as closed-ended questions. Open-ended questions generally help in obtaining valuable information, based on various individuals and the way the different way they interact with, or view, the system. These types of questions require the interviewee to explain or describe their thoughts, and cannot be simply answered with a "yes" or "no". Asking the interviewee what they like about the current

system or how they use it would be examples of open-ended questions. These types of questions can provide the consultant to further probe for more detail with follow up questions, in order to get more details. An example open-ended question would be “What are some of the problems you face on a daily basis?” Close-ended questions can also be useful, when the interviewer is looking for a specific answer. They can provide specific answers for the interviewee to choose from, in formats including true or false or multiple choice. Although close-ended questions do not provide as much detail as open-ended, they can be useful to cover more topics in a less amount of time.

An example of a close-ended question would be “How many telephone orders are received per day?” Once the questions have been established, it is a good practice to provide the questions to the interviewee prior to the interview, in the event that the interviewee needs to prepare. During the interview, the interviewer should obtain permission from the interviewee that recorders may be used, to ensure that if details are missed while taking notes can easily be retrieved. At the end of the interview, the results should be provided to the interviewee, for confirmation of their responses.

Group Interviews

Group interviews are similar to one-on-one interview, except there is more than one person being interviewed. Group interviews work well when the interviewees are at the same level or position. A group interview also has an advantage when there is a time constraint. More thoughts and discussion can be generated, as someone in the group may state or suggest an idea that may have been overlooked by others, which in turn can lead to a discussion or provide more information on a particular issue. The interviewer can gauge which issues are more generally agreed upon, and which are which issues differ. A major disadvantage can be scheduling the interview. When more than one person are involved, it may be difficult, or become time consuming, in establishing date and time that works well for all parties.

Questionnaires/Surveys

Questionnaires, or surveys, allow an analyst to collect information from many people in relatively short amount of time. This is especially helpful when stakeholders are spread out geographically, or there are dozens to hundreds of respondents whose input will be needed to help establish system requirements. When using questionnaires, the questions should be focused and organized by a feature or project objective. Questionnaires should not be too long, to ensure that users will complete them. When constructing the questionnaire, general guideline to determine the questions would be to ask “how, where, when, who, what, and why.”

For how: *“How will you use this feature?” “How might we meet this business need?” “How will we know this is complete?”*

For where: *“Where does the process start?” “Where would the user access this feature?” “Where would the results be visible?”*

For when: *“When will this feature be used?” “When will the feature fail?” “When will we be ready to start?”*
For who: *“Who will use this feature?” “Who will deliver the inputs for the feature?” “Who will deliver the outputs of the feature?”*
For what: *“What do I know about this feature?” “What does this feature need to do?” “What is the end result?” “What must happen next?”*

User Observation:
The direct approaches of interviewing and questionnaires provide valuable user feedback based on the questions asked of them; however, there are times when direct observation may be better suited in requirement gathering. To get a better understanding of a user in their in current work environment, the analyst may observe the user themselves. User observation is helpful in assisting the analyst by getting a full grasp of how the user interacts with the system, firsthand. When the objective is to improve a task, the analyst can observe the user and how their surroundings affect their interaction with the system. Sometimes stakeholders may find it difficult in explaining what exactly what their tasks consists of and what their requirements may be, observing the user in cases like these will help provide the requirements. User observation may also be useful in validating data that had been previously collected. Be it in cases where users provide misleading information, or cannot fully recollect all of their tasks in how they use the system.
User observation should be planned to ensure that all elements are constant surrounding the observation. This will assist in uncertainty, and the consultant can focus on the user and assist in knowing what to look for. The analyst will not be distracted and record, or note, irrelevant issues. The more useful information gathered, the less time it will take to the analyst to dissect and evaluate afterwards. Timing of the observation can also prove relevant when planning. For optimal results, the consultant should schedule three different periods of observation: low, normal, and peak times. This may prove helpful in because the user may interact with the system differently during different times. The consultant will take into consideration the differences in times time settings and use it to obtain to construct better requirements to assist in all three times. When observing the user, there are two approaches the consultant can take, passive or active. In passive observation, the consultant does not interact with the user while they are working. They simply observe and take notes. While in active observation, the consultant will ask the user questions during the session. Observations should always be done without bias, as in other requirement gathering techniques, the analyst must simply record what is presented to them, and avoid making comments on whether they believe what is correct or not. A checklist can provide to be useful, with key points already noted and the consultant verifying events on their list. For example, they can check if a user uses certain features, the frequency of events, triggers that cause different uses. Taking detailed notes is helpful in recording unexpected events. There may be events unknown to the analyst ahead of time, they can be captured by taking notes of the event and why it occurred. Video recorders may be used, but must always be approved with the user and their company. While there are many advantages to use observation, there are some disadvantages associated as well. As previously mentioned,

observations should take place during peak, normal, and low business times. However, it may still be difficult to capture enough information in one of these sessions. There may be the need for multiple sessions to verify that facts collected were constant, rather than isolated incidents. Analysts themselves can sometimes be biased in what they expected to see, and what they actually observed. Again, the focus is to simply collect the facts, not form any biased opinion when seeing the working environment. The users sometimes may not themselves provide an accurate depiction of their every day task and respond differently. Some users may become nervous, and not perform as they normally would. Other users may try and perform more better or worker harder when they know they are being observed. Such observations can inhibit the analyst to decipher what the true requirements actually should be.

Analyzing Existing Documents:

Analyzing existing documents can prove to be a useful technique in requirement gathering, on its own as well using it to supplement other techniques. Reviewing the current process and documentation can help the analyst understand the business, or system, and its current situation. Existing documentation will provide the analyst the titles and names of stakeholders who are involved with the system. This will help the analyst formulate questions for interviews or questionnaires to ask of stakeholders, in order to gain additional requirements. If an analyst is uncertain why certain procedures are in place, this can also help the analyst in asking these questions during interviews. When studying the requirements, the analysts may find problems that they may distinguish on their own. The analyst may find there was missing information in old documents. They may also find redundancy, in which steps are unnecessarily repeated. The consultant may look at old requirement documents and reuse of the requirements that may still be relevant, while discarding others that may be out of date. The reason why the current system is designed the way it is, which can suggest why certain features were left out. Principles and rules for the organization itself can be discovered by analyzing documents. Analyzing documents can be used as a supplement to information obtained from interviews, questionnaires, and observations. For example, if some of the interview answers are unclear, organizational documents may help in making sense of some of the interviewee's answers. Reviewing existing documents may also assist in understanding why a user performs certain tasks while observing them. A drawback to analyzing documents is that documents may be outdated, the analyst must confirm whether the documents are current or not. Another drawback to reviewing documents is it can be very time consuming, depending on the organization and the system. A system that interacts with many different facets of the business will have large amounts of documentation to review.

Joint Application Design/JAD:

The goal is to get the design right the first time, thereby reducing different iterations. JAD is usually conducted in at a location other than the place where the people involved work. This helps to reduce distractions. The participants of a JAD include: JAD session leader (also known as the facilitator), users, managers, sponsors, systems analysts, scribe, and other IS staff members. The facilitator, whom is usually trained in project management as well as system s analysis, manages

the entire process. They are responsible for keeping the group on the agenda and resolving conflicts. Facilitators do not contribute ideas; they solicit the ideas of the group. They also make the rules for the sessions, and can dismiss anyone who is troublesome to the process. Users involvement helps eliminate their frustration over the systems development process. The user is involved throughout a JAD session, and can make great contributions, as they will be the ones using the new system. Managers can provide more of insight into the organizational aspects of the system. The sponsor doesn't attend all JAD sessions, but usually just the beginning or end, as they do not contribute to ideas, they merely sponsor the process. Systems analysts are there to learn from users and managers. As in other requirements gathering techniques, they should not show bias. The scribe takes detailed notes during the session. The other IS staff members, such as programmers or database analysts attend to contribute technical aspects to the proposed ideas.
Some of the benefits of JAD include:

- Consolidation of months of work into a structured workshop
- Clarifies specification requirements in an environment of consensus
- Identifies open issues and parties responsible for their resolution
- Ties together the steps of the design process in a concise document
- Increases user satisfaction by directly involving users in the design process
- Builds commitment through the use of the executive sponsor
- Builds a sense of belonging and helps create a cohesive team through the physical and social setting

AJAD:

Although JAD is seen as a newer method, there are some instances where JAD has even developed further into AJAD (Automated JAD). A traditional JAD setting will be in a conference room of some type, with a "U" shaped table. There will be documents on the table, white boards used to write ideas, and agenda posted on the wall and a screen for projecting. An in AJAD session, users be still be seated in a "U" shaped table, but will have computers in front of them, instead of documents. Some of these settings will have desks that conceal their computer screens and keyboards beneath glass surfaces. The purpose is to promote team interaction and ensure that technology remains an enabler, not the focal point of decision making. AJAD uses groupware packages during their sessions. Groupware capabilities fall into three categories: information generation, information analysis/decision making, and information documentation. The Information generation capability supports simultaneous generation of information by a large group of people, which can prove to be less time consuming. This can also provide anonymity for participants who otherwise might not have wanted to share their ideas aloud. This feature also reduces the role of the facilitator as the intermediary, as fewer confrontations arise than when contributors are all debating ideas. Just as the information generation capability provided simultaneous generation of ideas, the information analysis/decision portion helps in simultaneous analysis of information. As users identify business activity and data, the matrix capability can be used to translate these lists into the two axes that the participants can use analyze the activity/data relationships. The groupware can use voting tools to generate a vote on for different system

features. Once voting is complete, the facilitator can display the results for everyone. Groupware's final category of information documentation is a self-documenting process. An instant record of all the information that the participants generate, all the analysis conducted, and all the decisions reached. Some tools may have the ability to show the different iterations.

Prototyping:

Prototyping is another form a contemporary requirement gathering method. Prototyping is iterative process that heavily involves the users to complete. The user provides the requirements, in which the analyst can plug in directly and show the user the outcome. Prototyping is dependent on user interaction and cannot be utilized as its own method of gathering requirements. The analyst must interview or perform some other form of requirement gathering to perform before they begin prototyping. However, prototyping is very effective in specifying requirements, because of how heavily involved the user is. The user will still be sitting side by side with the analyst, providing them requirements as the analysts enters them into a working system. This will allow the user to instantly see the outcome of their requirements. At this point the user may change some of their requirements. They may see that what they provided was not what they had in mind. A form may appear cluttered with information; at this point the user can go back and adjust their information. This may also be the case in when the user forgets important information; they may not realize it until they actually see a working version of the system. The user and analyst will continue to go through different iterations, until all specifications are complete. The last prototype will be used as a model to build the actual system. Some of the disadvantages of prototyping is the user will pay too much attention to details on the screens, rather than what the prototype is meant to communicate. Executives can grow impatient as they see a complete prototype, but will not understand why the finished system takes so long to complete

Conclusion

Requirement collection is the most important step of a project. If the project team fails to capture all the necessary requirements for a solution, the project will be running with a risk. This may lead to many disputes and disagreements in the future, and as a result, the business relationship can be severely damaged.

Therefore, take requirement collection as a key responsibility of the project team. Until the requirements are signed off, do not promise or comment on the nature of the solution.